PROJECT REPORT

Mini Hospital Management System

Architecture & Technical Documentation

Prepared by

Abhishek Kolpe

abhishekkolpe2004@gmail.com

1. Project Overview

The Mini Hospital Management System (HMS) is a web-based application designed to bridge the gap between healthcare professionals and patients. It streamlines appointment management by enabling doctors to set their availability and patients to seamlessly view and book time slots.

Key Objectives:

- Prevent double-booking (race conditions) during concurrent booking attempts.
- Notify actors instantly via email and calendar sync upon successful appointments.
- Separate core responsibilities (Web Application vs. Background Services) mimicking enterprise architectures.

2. System Architecture

The project employs a Service-Oriented Architecture (SOA) approach, separating concerns across a core Django monolith and a serverless email microservice.

2.1 The Core Django Monolith (hms_project/)

The main backend is structured into domain-driven Django applications:

- Accounts (accounts/) — Manages the custom User model, extending Django's AbstractUser with a role field ('doctor' vs 'patient') and OAuth tokens (google_token).
- Doctors (doctors/) — Manages AvailabilitySlot models where doctors define their working hours.
- Patients (patients/) — Manages Appointment records and implements the core transaction logic for slot booking.

- Core (core/) — Houses cross-cutting concerns: decorators for role-based access control, the email service caller, and Google Calendar helper.

2.2 The Serverless Email Service (serverless-email/)

To decouple email delivery from the main request-response cycle, email notifications are handled by a separate microservice:

- Built with AWS Lambda / Serverless Framework (running locally via serverless-offline).
- Receives a JSON payload via an HTTP POST request.
- Renders email templates (e.g., SIGNUP_WELCOME, BOOKING_CONFIRMATION).
- Dispatches SMTP emails via smtp.gmail.com using app passwords.

3. Database Design

The system uses PostgreSQL for data integrity and ACID compliance, which is vital for transactional booking logic.

User (Custom)	
`username`	Unique identifier for the user account
`email`	Contact email address
`role`	Determines access level: 'doctor' or 'patient'
`google_token`	Stored OAuth token for Google Calendar integration
Constraint: Role determines which dashboards and actions the user can access.	

AvailabilitySlot	
`doctor`	Foreign Key to User (doctor role)
`date`	The calendar date of the slot
`start_time`	Slot start time
`end_time`	Slot end time
`is_booked`	Boolean flag indicating if the slot has been taken
Constraint: unique_together on (doctor, date, start_time, end_time) prevents duplicate availability records.	

Appointment	
`patient`	Foreign Key to User (patient role)
`slot`	OneToOneField to AvailabilitySlot
`status`	Current booking status
`doctor_calendar_event_id`	Google Calendar event ID for doctor

Appointment	
`patient_calendar_event_id`	Google Calendar event ID for patient
Constraint: OneToOneField on slot ensures a single AvailabilitySlot can only be linked to ONE Appointment — a database-level safeguard against double-booking.	

4. Key Workflows & Pipelines

4.1 Authentication Pipeline

Session-based authentication is used via Django's auth middleware. Users hit the unified /accounts/login/ page and are redirected automatically based on their mapped role. The is_doctor and is_patient decorators intercept unauthorized traffic, enforcing Role-Based Access Control (RBAC) throughout the application.

4.2 Slot Booking Pipeline — The Critical Path

The most sensitive operation in the application is the book_slot view, which must handle scenarios where multiple patients attempt to book the same slot simultaneously.

Solution: Atomic Transactions with select_for_update()

```
with transaction.atomic():
    slot = AvailabilitySlot.objects.select_for_update().get(pk=slot_id)
    if slot.is_booked:
        # Race condition caught — safely rejected
        return error()
    slot.is_booked = True
    slot.save()
    Appointment.objects.create(...)
```

> **How select_for_update() Works**
> PostgreSQL applies an exclusive row-level lock. The first request locks the row. Concurrent requests queue behind it. Once unlocked, the second request reads is_booked=True and gracefully fails — preventing double-bookings at the database level.

4.3 Third-Party Integrations Pipeline

To avoid hanging the POST /book/ request, third-party integrations fire in background threads immediately after the database transaction concludes.

A. Google Calendar Sync (core/calendar_helper.py)

- A background thread retrieves the stored google_token for the appropriate user.
- Re-constructs Google API credentials and injects a new .insert() event payload.
- Captures the generated Event IDs and saves them back to the Appointment record.

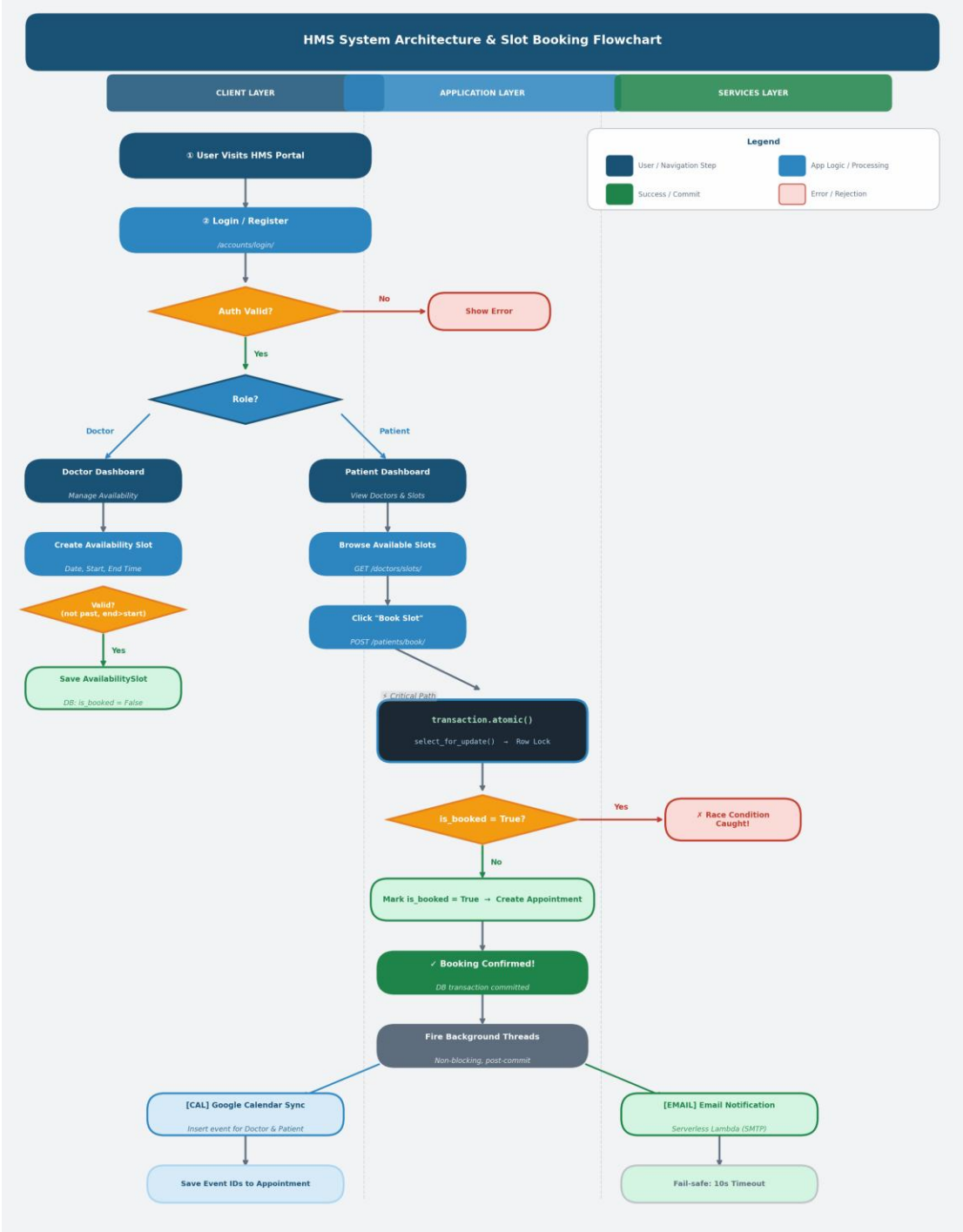
B. Email Notification Sync (core/email_service.py)

- A background thread executes a requests.post() call to the serverless-offline Lambda endpoint.
- Configured with a 10-second timeout — if the endpoint is down, the parent request is entirely unaffected.
- Errors are caught and logged, ensuring graceful degradation without impacting the user experience.

5. UI/UX & Asset Architecture

Design Principle	Implementation
No Framework Bloat	Vanilla CSS with custom properties (var(--primary), var(--bg-card)) for a premium dark-mode theme.
Feedback Loops	messages.success and messages.error popups guide users through all actions seamlessly.
Animations	Subtle pop-in scale transforms on booking confirmation interfaces for a modern feel.
Responsive Design	CSS variables ensure consistent, adaptive styling across all device sizes.

The diagram below illustrates the complete flow from user authentication through role-based routing to the critical slot booking pipeline, including background service integrations.



6. Security Considerations

- CSRF Protection enabled globally across all POST forms.
- Data Validation — the AvailabilityForm strictly prohibits creating slots in the past or with end times before start times.

- Token Storage — external API credentials (Google OAuth JSON) are stored physically outside the version-controlled codebase, keeping secrets out of git.

7. Future Enhancements

Given more time, the next phases of development would include:

1. Paginating the Doctor/Slot lists using AJAX for better performance at scale.
2. Integrating a task queue like Celery with Redis instead of native Python threading to ensure message delivery reliability.
3. Implementing caching for static assets and doctor lists to enhance response times.